BÀI THỰC HÀNH CHƯƠNG 6

I. Sử dụng tập tin tĩnh lưu trên một App

A. Truy xuất đến một tập tin tĩnh trong App

Bài này sẽ thực hành thêm logo vào app **landing**. Việc này thực hiện bằng cách đặt tập tin **logo.png** vào thư mục **static** trong thư mục app **landing**.

1. Đầu tiên, ta tạo mới một dự án Django. Có thể sử dụng lại môi trường ảo (virtual environment) của dự án **Bookr**. *Trường hợp muốn tạo môi trường ảo mới thì lưu ý cài đặt gói Django.*

2. Chạy **django-admin** để tạo mới dự án tên là **business_site**:

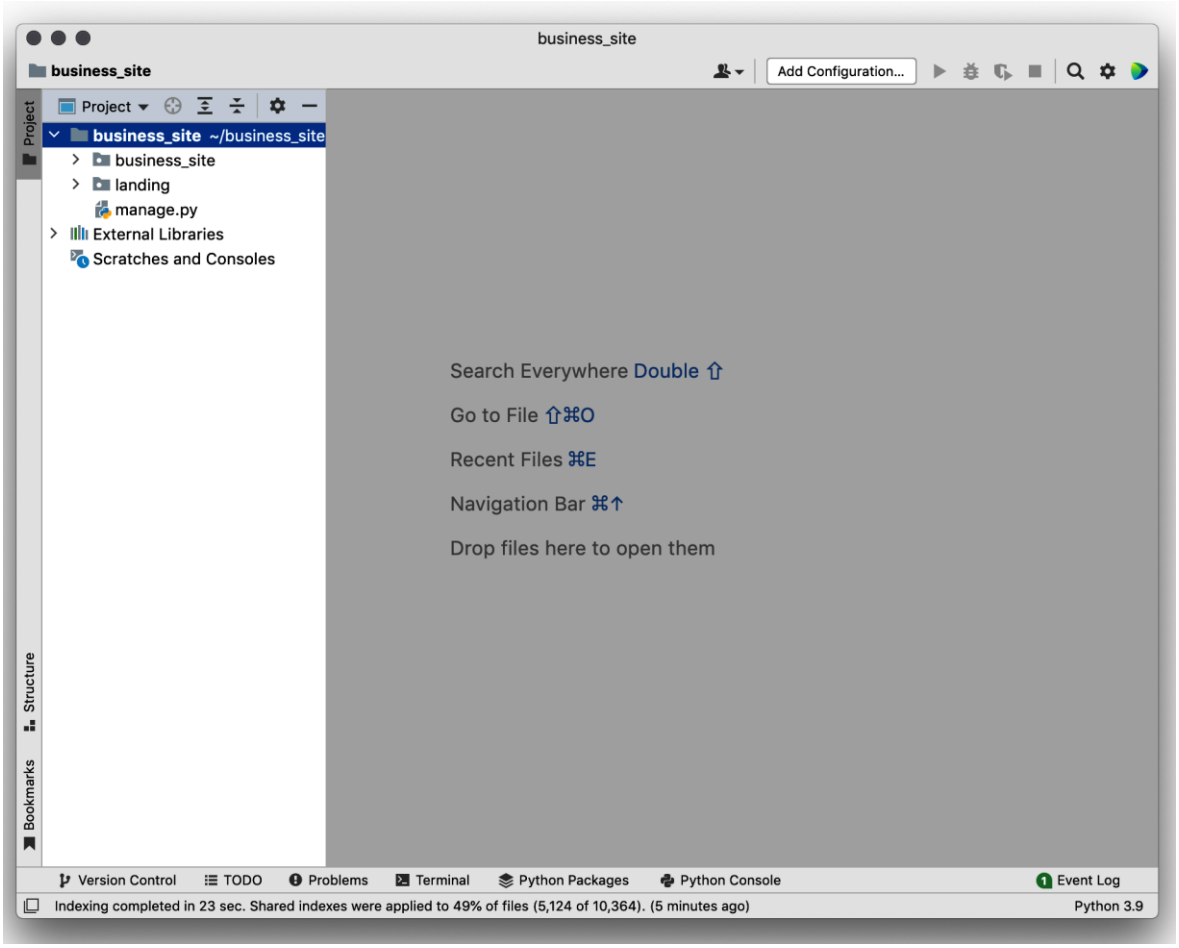
```
django-admin startproject business_site
```

Lệnh nào không xuất nội dung nào nhưng sẽ tạo một thư mục cho dự án có tên là **business_site**.

3. Tạo mới một app tên là **landing** trong dự án mới này. Lưu ý dùng lệnh **CD** để chuyển sang thư mục **business_site** trước khi chạy lệnh **manage.py startapp**:

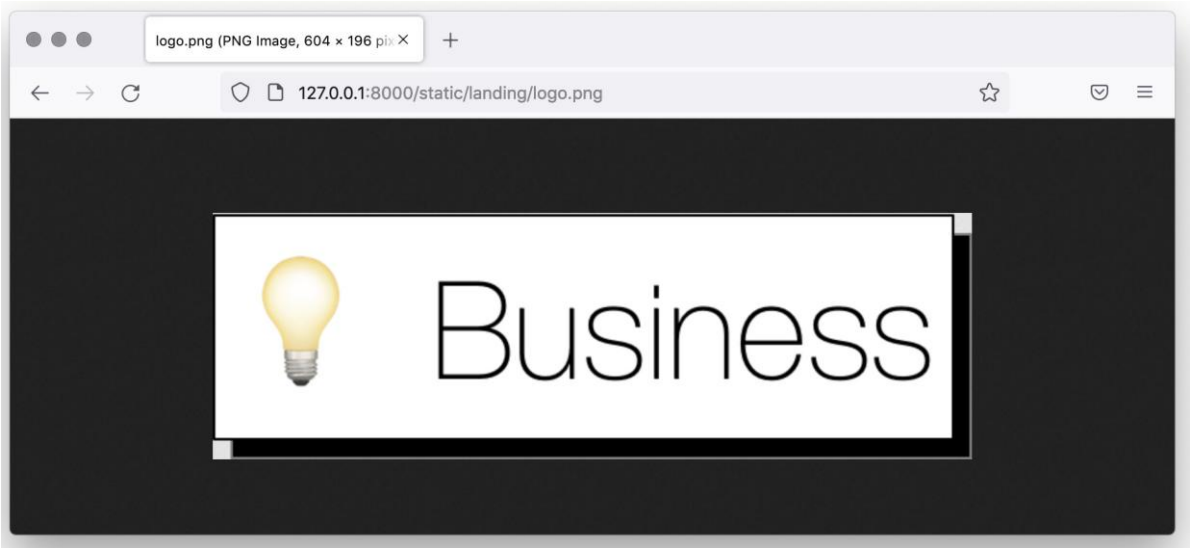
```
python manage.py startapp landing
```

Sau khi thực thi lệnh trên, tổ chức thư mục của dự án **business_site** sẽ như **Hình I-1.**



Hình I-1. Tổ chức thư mục dự án business_site

4. Tạo mới một cấu hình chạy máy chủ web **manage.py runserver** theo **Phần III.B** của **Bài thực hành Chương 1**.
5. Mở tập tin **settings.py** trong thư mục dự án **business_site** và thêm app **landing** vào biến **INSTALLED_APPS**.
6. Tạo mới 1 thư mục con **static** trong thư mục **landing** bằng cách chọn menu **New → Directory** trên **PyCharm**.
7. Lặp lại **Bước 6** để tạo mới thư mục con **landing** trong thư mục **static** vừa tạo.
8. Tải xuống tập tin **logo.png** từ đường link sau https://raw.githubusercontent.com/PacktPublishing/Web-Development-with-Django-Second-Edition/main/Chapter05/Exercise5.01/business_site/landing/static/landing/logo.png và lưu vào thư mục **landing/static/landing**.
9. Khởi chạy trình chủ phát triển và mở link sau http://127.0.0.1:8000/static/landing/logo.png. Kết quả hình sẽ hiển thị như trong **Hình I-2**.



*Hình I-2. Hình từ tập tin **logo.png** trên website **business_site***

B. Sử dụng thẻ template `static`

Bài này sẽ thực hành tạo một tập tin template **index.html** cho trang chính của website. Sau đó sẽ thêm logo vào trang này sử dụng thẻ template **{% static %}**

1. Trên **PyCharm**, tại dự án **business_site**, tạo mới thư mục **templates** nằm trong thư mục **landing**.
2. Bấm phải chuột vào thư mục **templates** vừa tạo và chọn **New → HTML File**, rồi chọn tiếp **HTML 5** và nhập tên tập tin là **index.html** để tạo mới tập tin template **index.html**.
3. Mở tập tin **index.html** và tiến hành thêm đoạn mã để *tải gói **static** về template* trước khi sử dụng bằng cách chèn thêm 1 dòng mã sau vào **sau dòng <!DOCTYPE html>** :

```
{% load static %}
```

4. Thực hiện điều chỉnh giao diện trang template **index.html** cho đẹp hơn như:
   - Thêm tiêu đề vào thẻ <title>: `<title>Business Site</title>`
   - Thêm tiêu đề trang với thẻ <h1>: `<h1>Welcome to my Business Site</h1>`

5. Bên dưới tiêu đề trang, thêm hình ảnh logo. Sử dụng thẻ template **static** để đặt giá trị cho thuộc tính source của thẻ **<img>,** ta sử dụng tập tin **logo.png** đã dùng ở bài thực hành trước.

```
<img src="{% static 'landing/logo.png' %}">
```

6. Cuối cùng, thêm một thẻ **<p>** dưới thẻ **<img>** để xuất ra một đoạn nội dung:

```
<p>Welcome to the site for my Business. For all your Business
needs!</p>
```

Lưu lại tập tin.

7. Tiếp theo, tạo một **URL** để xuất ra template. Ta có thể sử dụng lớp **TemplateView** sẵn có để xuất nội dung template mà *không cần tạo một hàm View*. Mở tập tin **urls.py** trong thư mục **business_site.** Thêm dòng mã sau để **import** lớp **TemplateView**:

```
from django.views.generic import TemplateView
```

Ta cũng có thể bỏ đoạn mã import gói admin do không cần sử dụng:

```
from django.contrib import admin
```

8. Thêm môt ánh xạ URL để mở trang template vừa tạo bằng cách thêm một đường dẫn vào biến **urlpatterns**:

```
urlpatterns = [
    path("", TemplateView.as_view(template_name="index.html")),
]
```

Trong khai báo đường dẫn trên, ta sử dụng phương thức **as_view** của lớp **TemplateView** với tham số **template_name** là tập tin template **index.html** để xuất trực tiếp nội dung trang template mà không thực hiện thêm thao tác xử lý nào khác.

9. Khởi chạy trình chủ phát triển web và mở trang http://127.0.0.1:8000/ . Nội dung của trang **index.html** sẽ như **Hình I-3**.

Hình I-3.Giao diện trang chính của business_site sử dụng template index.html

II. Truy xuất đến tập tin từ một thư mục tĩnh

Trong bài này, ta sẽ thực hành thiết đặt để cho phép truy xuất đến các tập tin tĩnh từ một thư mục cụ thể, sau đó dùng thẻ template **static** để thêm tập tin vào template. Bài này sẽ tiếp tục sử dụng dự án **business_site**.

1. Mở dự án **business_site** trên **PyCharm**. Tạo mới một thư mục con **static** trong thư mục gốc của dự án này (trong thư mục **business_site gốc**).
2. Thêm mới một tập tin **main.css** trong thư mục **static** vừa tạo bằng cách bấm phải chuột vào thư mục này và chọn **New → File** rồi nhập tên tập tin là **main.css**.
3. Trong tập tin **main.css**, thêm một số quy tắc để canh giữa cho văn bản và đặt màu nền văn bản như sau:

```
body {
    font-family: Arial, sans-serif;
    text-align: center;
    background-color: #f0f0f0;
}
```

   Lưu lại tập tin.

4. Mở tập tin **business_site/settings.py** và khai báo thư mục **static** tạo ở trên là thư mục tĩnh bằng cách thiết đặt giá trị cho biến **STATICFILES_DIRS** như sau:

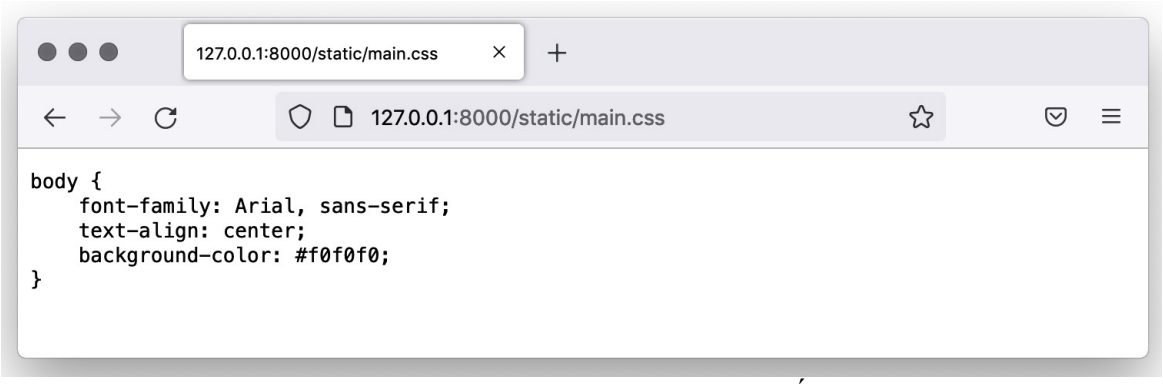
```
STATICFILES_DIRS = [BASE_DIR / "static"]
```

   Do *biến STATICFILES_DIRS chưa được khai báo* trong tập tin **settings.py** nên ta cần thêm nguyên dòng mã trên. Trong đó, biến **BASE_DIR** chứa đường dẫn đến thư mục gốc của dự án.

5. Chạy trình chủ phát triển **Django** và kiểm tra việc cấu hình đã đúng chưa bằng cách truy xuất trực tiếp đến tập tin **main.css** trong thư mục static bằng URL sau:

```
http://127.0.0.1:8000/static/main.css
```

   Nếu cấu hình đúng thì kết quả hiển thị trên trình duyệt sẽ như **Hình II-1**.

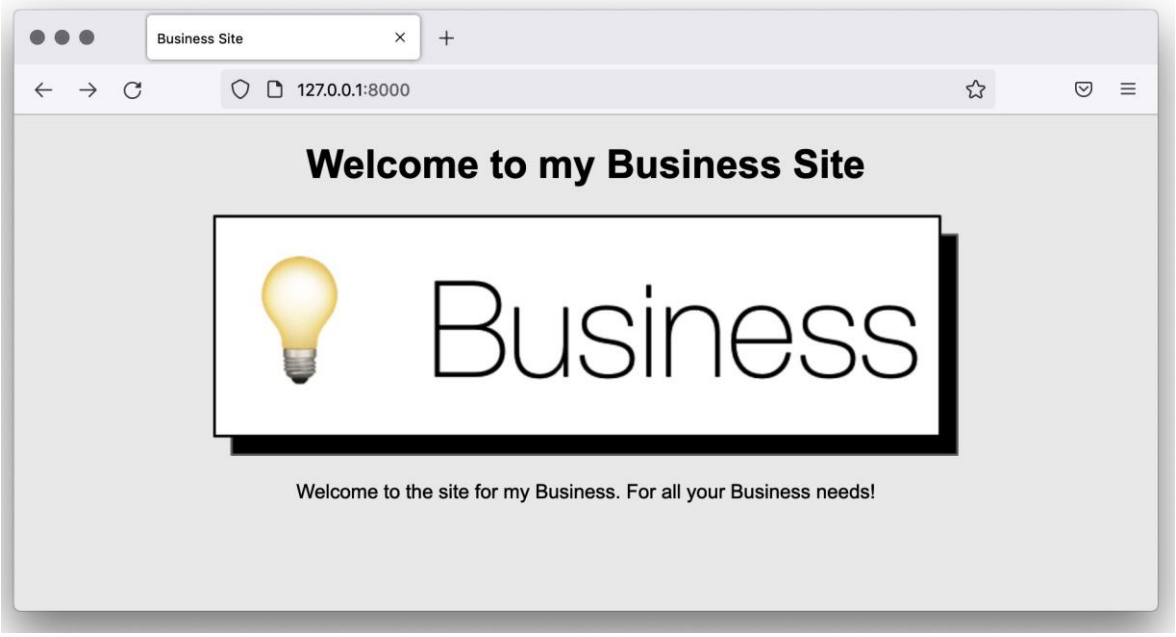


*Hình II-1. Tập tin nội dung tập tin **main.css** truy xuất từ thư mục **static***

6. Bây giờ, ta cần thêm tập tin **main.css** vào template. Mở tập tin **index.html** trong thư mục **templates**. Thêm một thẻ **<link>** vào ngay trước **thẻ đóng </head>** để tải về tập tin **main.css** như sau:

```
<link rel="stylesheet" href="{% static 'main.css' %}">
```

7. Mở lại trang chính của **business_site** theo URL http://127.0.0.1:8000/, kết quả xuất hiện trên trình duyệt sẽ như **Hình II-2.**



*Hình II-2. CSS đã được thêm và áp dụng vào trang chính của **business_site***

III. Một số tính năng mở rộng trong quản lý tập tin tĩnh (Không bắt buộc)

A. Thu thập các tập tin tĩnh để phục vụ triển khai ứng dụng web

Để triển khai các ứng dụng web thì một công việc quan trọng là cần tập hợp tất cả các tập tin tĩnh để sao chép lên máy chủ ứng dụng web mới. Việc này có thể được thực hiện bằng lệnh **collectstatic**. Sinh viên thực hành sử dụng thử lệnh này theo hướng dẫn trong bài thực hành **Exercise 5.04 – collecting static files for production** tại trang **235-236** của **Giáo trình**.

B. Tìm kiếm tập tin tĩnh sử dụng lệnh findstatic

Lệnh **findstatic** rất hữu ích khi cần tìm kiếm các tập tin tĩnh trong một dự án lớn. Sinh viên khám phá thêm các tùy chọn này theo bài thực hành **Exercise 5.05 – finding files using findstatic** tại trang **240-242** của **Giáo trình.**

C. Các tính năng của bộ lưu trữ ManifestFilesStorage

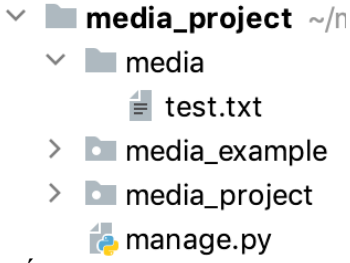
Việc kiểm soát lưu đệm là cần thiết khi ứng dụng web có các tập tin tĩnh được cập nhật thường xuyên, đặc biệt là trong quá trình phát triển ứng dụng. Lớp **ManifestFilesStorage** hỗ trợ việc kiểm soát các phiên bản tập tin tĩnh bằng cách bổ sung thêm đoạn mã băm vào tên tập tin. Sinh viên có thể khám phá thêm các tính năng của lớp này trong bài thực hành **Exercise 5.06 – exploring the ManifestFilesStorage storage engine** tại trang **245-248** của **Giáo trình**.

IV. Thiết đặt thư mục lưu trữ tập tin và truy xuất đến tập tin từ Template

A. Cấu hình thư mục lưu trữ tập tin

Bài thực hành này sẽ thiết lập một dự án Django để thực hành đến cuối chương này. Sau đó, sinh viên sẽ cấu hình để có thể làm việc việc với tập tin đa phương tiện, bắt đầu từ tạo mới thư mục chứa tập tin đến cấu hình các biến **MEDIA_ROOT** và **MEDIA_URL** và ánh xạ URL cho biến này.

1. Tiến hành tạo mới dự án tên là **media_project** và app **media_example** với 2 lệnh sau:
```
django-admin startproject media_project
python3 manage.py startapp media_example
```
2. Thiết đặt cấu hình để chạy máy chủ web với lệnh **runserver** bằng nút bấm trên thanh công cụ của **PyCharm** theo **Bài thực hành 3.B ở Chương 1**.
3. Tạo mới một thư mục tên là **media** trong thư mục gốc **media_project** và tạo mới một tập tin trong thư mục mới tạo này với tên là **test.txt**. Cấu trúc thư mục dự án sẽ như sau:



Hình IV-1. Cấu trúc thư mục dự án media_project

4. Mở tập tin **test.txt** và nhập vào dòng "**Hello, world!**". Lưu lại tập tin.
5. Mở tập tin **settings.py** trong thưc mục con **media_project**. Thêm vào cuối tập tin một dòng khai báo giá trị cho **MEDIA_ROOT** là đường dẫn đến thư mục **media** vừa tạo ở **Bước 3**:
```
MEDIA_ROOT = BASE_DIR / "media"
```
6. Tiếp tục thêm một dòng khai báo giá trị cho **MEDIA_ROOT** vào dưới dòng khai báo ở **Bước 5**:
```
MEDIA_URL = "/media/"
```
7. Mở tập tin **urls.py** nằm trong thư mục con **media_project** và 2 thêm dòng **import** gói **settings** và **static**. Sau đó thêm dòng khai báo sử dụng URL đến thư mục **media** trong chế độ **DEBUG**:
```
from django.contrib import admin
from django.urls import path
from django.conf import settings
from django.conf.urls.static import static
urlpatterns = [
    path("admin/", admin.site.urls),
]
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
    document_root=settings.MEDIA_ROOT)
```
8. Khởi chạy máy chủ phát triển Django và mở tập tin **test.txt** với URL sau: **http://127.0.0.1:8000/media/test.txt**. Kết quả sẽ như **Hình IV-1**.

Hình IV-2. Kết quả trang xuất nội dung tập tin test.txt sau khi cấu hình thành công

B. Các thiết đặt Template và sử dụng MEDIA_URL trong template

Bài thực hành sẽ tiếp tục trong dự án **media_project**. Sinh viên sẽ cấu hình để Django tự động thêm biến **MEDIA_URL** vào tất cả các **template**. Việc này được thực hiện bằng cách thêm một **django.template.context_processors.media** vào biến **TEMPLATES**. Sau đó, thêm mới một template sử dụng biến mới khai báo trên và một view để xuất ra trên template.

1. Trong **Pycharm**, mở tập tin **settings.py** và thêm app **media_example** vào biến **INSTALLED_APPS**:

```
INSTALLED_APPS = [
    ……
    "media_example",
]
```

2. Thêm đoạn mã "**django.template.context_processors.media**" vào trong mục **OPTIONS** của biến **TEMPLATES**, nội dung khai báo biến **TEMPLATES** sẽ như sau:

```
TEMPLATES = [
{
    "BACKEND":
        "django.template.backends.django.DjangoTemplates",
    "DIRS": [],
    "APP_DIRS": True,
    "OPTIONS": {
        "context_processors": [
            "django.template.context_processors.debug",
            "django.template.context_processors.request",
            "django.contrib.auth.context_processors.auth",
            "django.contrib.messages.context_processors.
messages",
            "django.template.context_processors.media",
        ],
    },
},
]
```

3. Mở tập tin **views.py** trong thư mục **media_example** và tạo một view mới tên là **media_example**. View này sẽ gọi hàm **render()** một template tên là **media_example.html.** Dòng mã thêm vào sẽ như sau:

```
def media_example(request):
    return render(request, "media-example.html")
```

4. Ta cần thêm một ánh xạ URL đến view **media_example**. Mở tập tin **urls.py** trong thư mục **media_project** con. Thêm đoạn mã để **import** tập tin views trong app **media_example**, sau đó thêm ánh xạ URL trong biến **urlpatterns**:

```
import media_example.views
urlpatterns = [
    path("admin/", admin.site.urls),
    path("media-example/", media_example.views.media_example)
]
```

5. Tạo thư mục **templates** trong thư mục app **media_example**. Sau đó, tạo một tập tin **HTML 5** trong thư mục **templates** vừa tạo với tên là **media-example.html**.
6. Thêm một đường link vào trong thẻ **<body>** của tập tin **media-example** để truy cập đến tập tin **test.txt** đã tạo ở phần thực hành trước:

```
<body>
    <a href="{{ MEDIA_URL }}test.txt">Test Text File</a>
</body>
```

7. Khởi chạy máy chủ phát triển Django và mở trang **http://127.0.0.1:8000/media-example/**. Kết quả sẽ như **Hình IV-3**. Say khi bấm chuột vào đường link Test Text File, sẽ mở ra cập tin **test.txt** như trong **Hình IV-2**.

Hình IV-3. Trang media-example

V. Tải lên và tải xuống tập tin

A. Sử dụng form HTML

Trong bài thực hành này, ta sẽ thêm một form với một hộp nhập **FileField** vào trang media-**example.html**. Hộp này sẽ cho phép tải một tập tin view **media-example** sử dụng trình duyệt web. Ta cũng có thể cập nhật lại view **media-example** để lưu tập tin vào thư mục **MEDIA_ROOT** để có thể tải về.

1. Trong **PyCharm**, mở tập tin template **media-example.html** trong thư mục **templates**. Trong thẻ **<body>,** loại bỏ thẻ **<a>** đã thêm vào ở **Bài thực hành IV.B** và thay bằng một form với **method** là "**post**" và **enctype** là "**multipart/form-data**":

```
</head>
<body>
    <form method="post" enctype="multipart/form-data">
</form>
</body>
```

2. Chèn thẻ template **{% csrf_token %}** vào trong thẻ **<form>.**
3. Sau thẻ {% csrf_token %}, thêm một thẻ **<input>** với **type** là "**file**" và **name** là "**file_upload**"

```
<input type="file" name="file_upload">
```

4. Cuối cung, trước thẻ đóng **</form>,** thêm một thẻ **<button>** với **type** là "**submit**" và nhãn là "**Submit**":

```
<button type="submit">Submit</button>
```

Sau khi thêm, thẻ **<form>** sẽ có nội dung như sau:

```
<body>
    <form method="post" enctype="multipart/form-data">
        {% csrf_token %}
        <input type="file" name="file_upload">
        <button type="submit">Submit</button>
    </form>
</body>
```

5. Mở tập tin **views.py** của app **media_example** và thêm đoạn mã để lưu tập tin tải lên vào thư mục **MEDIA_ROOT** đĩa. Để sử dụng biến **MEDIA_ROOT**, ta cần **import** gói **settings** ở đầu tập tin **views.py** như sau:

```
from django.conf import settings
```

6. Tập tin tải lên chỉ có thể được lưu nếu phương thức gửi là **POST** nên cần kiểm tra phương thức gửi bằng đoạn mã sau:

```
def media_example(request):
    if request.method == "POST":
```

7. Trong khối **if**, sinh đường dẫn lưu tập tin vàng cách nối tên của tập tin tải lên vào tên thư mục **MEDIA_ROOT**. Sau đó, mở tập tin ở chế độ wb và duyệt qua tập tin tải lên bằng phương thức **chunks().** Cuối cùng, lưu từng đoạn vào tập tin đã mở.

```
def media_example(request):
    if request.method == 'POST':
        save_path = settings.MEDIA_ROOT /
        request.FILES["file_upload"].name
```

```
        with open(save_path, "wb") as output_file:
            for chunk in request.FILES["file_upload"].chunks():
            output_file.write(chunk)
    return render(request, "media-example.html")
```

Lưu ý: Có thể truy cập đến tập tin tải lên và thông tin của nó từ biến **request.FILES** kèm với khóa là tên của hộp nhập tải tập tin lên (là **file_upload**).

8. Khởi chạy trình chủ phát triển Django và truy cập http://127.0.0.1:8000/media-example/. Ta sẽ mở ra trang web với một hộp chọn tập tin tải lên và nút **Submit** như **Hình V-1**.

Hình V-1. Form tải tập tin lên

Bấm chuột vào nút **Browse…** và chọn tập tin để tải lên. Tên của tập tin sẽ xuất hiện trong hộp chọn tập tin tải lên. Sau đó nhấn nút **Submit**.

Trang web sẽ được tải lại và form lại xuất hiện với hộp chọn tập tin tải lên trống. Đây là kết quả đúng, thực tế tập tin đã được tải lên và lưu trên thư mục **MEDIA_ROOT**.

9. Thử mở tập tin vừa tải lên (ví dụ là cover.jpg) bằng link **http://127.0.0.1:8000/media/cover.jpg**. Kết quả sẽ hiển thị nội dung tập tin đã tải lên.

B. Sử dụng Django forms

Trong bài thực hành này, ta sẽ tạo một form bằng thư viện **Django forms** với trường **FileField** cho phép ta có thể sử dụng các tính năng kiểm tra tính hợp lệ của dữ liệu nhập.

1. Trong **PyCharm**, tạo mới một tập tin là **forms.py** trong thư mục **media_example**. Ở đầu tập tin này **import** thư viện **Django forms**:

```
from django import forms
```

Sau đó, tạo một lớp con của lớp **forms.Form** tên là **UploadForm**. Thêm vào lớp này một trường **FileField** tên là **file_upload**. Đoạn mã của lớp này sẽ như sau:

```
class UploadForm(forms.Form):
    file_upload = forms.FileField()
```

2. Mở tập tin **views.py** của app **form_example**. Ở phần đầu tập tin, ngay dưới các dòng **import** hiện có, thêm một dòng mã để **import** lớp **UploadForm:**

```
from .forms import UploadForm
```

3. Thêm đoạn mã sau vào ngay sau dòng "**if request.method== "POST"** để khởi tạo đối tượng **UploadForm**:

```
form = UploadForm(request.POST, request.FILES)
```

Do form này có trường FileField nên cần truyền thêm tham số **request.FILES** để có thể truy xuất vào tập tin tải lên.

4. Điều chỉnh lại đoạn mã **save_path = …** thành một khối chỉ thực thi sau khi kiểm tra tính hợp lệ của dữ liệu tải lên từ form như sau:

```
if form.is_valid():
    save_path = os.path.join(settings.MEDIA_ROOT,
    request.FILES["file_upload"].name)

    with open(save_path, "wb") as output_file:
        for chunk in request.FILES["file_upload"].chunks():
            output_file.write(chunk)
```

5. Thay thế đoạn mã **request.FILES["file_upload"]** bằng **form.cleaned_data["file_upload"]**

```
    with open(save_path, "wb") as output_file:
        for chunk in form.cleaned_data["file_upload"].chunks():
            output_file.write(chunk)
```

6. Cuối dùng, thêm một nhánh để xử lý trường hợp các yêu cầu gửi đến không phải bằng phương thức **POST**. Trường hợp này chỉ tạo một form mới hoàn toàn:

```
if request.method == "POST":
    …
else:
    form = UploadForm()
```

7. Thêm một biến ngữ cảnh (**context**) với tên là "form" và giá trị là form cho hàm **render():**

```
return render(request, "media-example.html", {"form":form})
```

8. Cuối cùng, mở tập tin template **media-example.html**, xóa bỏ đoạn mã chèn các hộp nhập thủ công bằng mã HTML và thay thế bằng form xuất bởi phương thức as_p.

```
<body>
    <form method="post" enctype="multipart/form-data">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Submit</button>
    </form>
</body>
```

9. Khởi chạy trình chủ phát triển Django và truy cập **http://127.0.0.1:8000/media-example/**. Kết quả sẽ như **Hình V-2**.

Hình V-2. Form tải tập tin lên định nghĩa bởi Django forms

10. Do hộp chọn tập tin tải lên sinh bởi Django forms nên có sẵn tính năng tự động kiểm tra tính hợp lệ. Nếu ta cố submit form mà không chọn tập tin tải lên, trình duyệt sẽ ngăn lại và hiển thị một lỗi như trong Hình V-3.

Hình V-3. Việc submit form bị chặn lại bởi trình duyệt web

11. Cuối cùng, thực hiện tải thử một tập tin lên. Sau khi tập tin tải lên, ta có thể truy cập đến tập tin sử dụng **MEDIA_URL**, như **Bước 8 và 9** ở **Bài thực hành 5.A.**

C. Tải lên tập tin hình ảnh sử dụng Django forms

Thực hành sử dụng trường ImageField thay thế cho trường FormField ở bài thực hành trên và thực hiện thay đổi kích thước hình ảnh theo hướng dẫn tại mục **Exercise 8.05 – image uploads using Django forms** ở trang **394** của **Giáo trình**.

VI. FileField và ImageField trong Model

Bài thực hành sẽ tạo một model với các trường **FileField** và **ImageField**. Sau đó, ta sẽ phải tạo bảng dữ liệu tương ứng trên CSDL từ model vừa tạo. Tiếp đó, ta sẽ thay đổi lớp **UploadForm** để có 2 trường **FileField** và **ImageField** và cập nhật view để lưu các tập tin tải lên vào ổ đĩa. Cuối cùng ta sẽ thêm một thẻ **<img>** vào template để hiển thị hình ảnh tải lên trước đó.

1. Trong **PyCharm**, mở tập tin **models.py** của app **media_example**. Tạo mới một model tên là **ExampleModel** với 2 trường **FileField** và **ImageField** có tên lần lượt là **image_field** và **file_field**. Thuộc tính **upload_to** của 2 trường này lần lượt là **files/** và **images/**. Đoạn mã định nghĩa model sẽ như sau:
```
class ExampleModel(models.Model):
    image_field = models.ImageField(upload_to="images/")
    file_field = models.FileField(upload_to="files/")
```
2. Mở **terminal** và truy cập đến thư mục **media_project**. Lưu ý, có thể sử dụng môi trường ảo của dự án **bookr**. Chạy lệnh **makemigrations** để sinh mã tạo bảng dữ liệu tương ứng với model vừa tạo.
```
python manage.py makemigrations
```
3. Thực hiện lện **migrate** để tạo bảng dữ liệu:
```
python3 manage.py migrate
```
4. Mở tập tin **forms.py** của app **media_example**. Đổi tên trường **ImageField** hiện có từ **file_upload** thành **image_upload**. Sau đó, thêm một trường **FileField** với tên **file_upload**. Đoạn mã định nghĩa **UploadForm** sẽ như sau:
```
class UploadForm(forms.Form):
    image_upload = forms.ImageField()
    file_upload = forms.FileField()
```
5. Mở tập tin **views.py** của app **media_example**. Trước tiên, import model **ExampleModel** như sau:
```
from .models import ExampleModel
```
6. Trong view **media_example**, đặt giá trị mặc định cho đối tượng **instance** là **None**.
```
def media_example(request):
    instance = None
```
7. Ta có thể xóa bỏ hoàn toàn nội dung của nhánh **form.is_valid()** do không cần nữa. Thay vào đó, ứng dụng tự động lưu khi đối tượng lưu dữ liệu vào đĩa. Ta cần khởi tạo một đối tượng **ExampleModel** và đặt giá trị trường **FileField** và **ImageField** là tên tập tin tải lên. Thêm đoạn mã sau vào dưới dòng **if form.is_valid()**:
```
    instance = ExampleModel()
    instance.image_field = form.cleaned_data["image_upload"]
    instance.file_field = form.cleaned_data["file_upload"]
    instance.save()
```
8. Truyền đối tượng vừa tạo sang template qua biến ngữ của của hàm **render()** với khóa là **instance**:
```
return render(request, "media-example.html", {"form": form,
"instance": instance})
```

9. Mở tập tin **media_example.html** và thêm một thẻ **<img>** để hiển thị hình ảnh tải lên lần cuối. Dưới thẻ đóng **</form>,** thêm một thẻ template **if** và kiểm tra biến **instance** đã được truyền chưa. Nếu đã được truyền, hiển thị thẻ **<img>** với thuộc tính **src** là **instance.image_field.url**

```
{% if instance %}
    <img src="{{ instance.image_field.url }}">
{% endif %}
```

10. Khởi chạy trình chủ phát triển Django và mở http://127.0.0.1:8000/media-example/. Form mở ra sẽ có dạng như Hình VI-1.

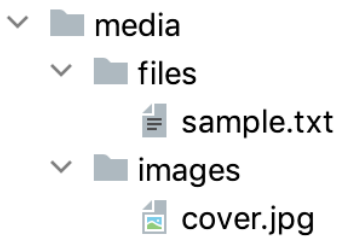
Hình VI-1. UploadForm với hai nhãn

11. Chọn một tập tin cho mỗi trường – đối với trường **ImageField**, ta cần phải chọn một hình ảnh, lưu ý là trường **FileField** cho phép tải lên nhiều loại tập tin khác. Kết quả như **Hình VI-2.**

Hình VI-2. Hai trường ImageField và FileField với tên tập tin đã được chọn

Sau khi tải tập tin lên, trang web sẽ tự tải lại và phần hình ảnh cuối cùng ta tải lên sẽ hiển thị chi tiết.

12. Tổ chức thư mục media như Hình VI-3.



Hình VI-3. Tập tin đã được tạo bởi Django

VII. Tải tập tin và hình ảnh sử dung ModelForm

Sinh viên tự thực hành theo hướng dẫn tại bài Exe**rcise 8.07 – file and image uploads using ModelForm** tại trang **416-419** của **Giáo trình.**